

ARYABHATTA KNOWLEDGE UNIVERSITY, PATNA

Bachelor of Computer Applications — Semester III

JAVA PROGRAMMING

Paper Code	BCA-303
Full Marks	60
Time	3 Hours
PYQ Years	2014, 2017, 2018, 2019, 2020, 2022, 2024, 2025

CONTENTS

Unit	Topic	MCQs	4-Mark	12-Mark
1	Java Evolution & Overview	6	4	2
2	Constants, Variables & Data Types	6	4	1
3	Operators, Branching & Looping	6	4	2
4	Classes, Objects, Methods, Inheritance	6	5	3
5	Arrays, Threads, Exception, Applets	6	4	3

VVI — MOST IMPORTANT TOPICS (Study These First)

- ★ Exception Handling (try-catch-finally) — Repeated 5+ times
- ★ Applet Life Cycle (init, start, paint, stop, destroy) — Repeated 6 times
- ★ Inheritance (types + single inheritance program)
- ★ JVM and Bytecode explanation
- ★ Multithreading (Thread lifecycle + program)
- ★ Constructor and Constructor Overloading
- ★ OOP Concepts (Encapsulation, Inheritance, Polymorphism, Abstraction)

UNIT 1 — Java Evolution & Overview

JVM · Bytecode · Features of Java · Diff from C/C++ · Simple Java Program · Tokens

SECTION A — MCQ Questions (2 Marks Each)

Q1. What does JVM stand for? [2024]

- (A) Java Visual Machine
- (B) Java Virtual Machine ✓**
- (C) Java Valid Module
- (D) Java Verbose Machine

Explanation: JVM = Java Virtual Machine. It runs bytecode and makes Java platform-independent.

Q2. Which command COMPILES a Java program? [2019]

- (A) java
- (B) javac ✓**
- (C) run
- (D) execute

Explanation: "javac FileName.java" compiles source code. "java ClassName" runs the compiled bytecode.

Q3. `int a=5; int b=6; Y = b++ + ++a; Find Y.` [2014]

- (A) 11
- (B) 12 ✓**
- (C) 13
- (D) 14

Explanation: `b++` uses `b=6` first (post). `++a` makes `a=6` first (pre). $Y = 6 + 6 = 12$.

Q4. Java bytecode files use which extension? [2017]

- (A) .java
- (B) .exe
- (C) .class ✓**
- (D) .byte

Explanation: .class = compiled bytecode file. .java = source code file.

Q5. Java is platform-independent because of: [2017]

- (A) Compiler
- (B) JVM and Bytecode ✓**
- (C) Syntax
- (D) IDE

Explanation: JVM reads the same .class bytecode on any OS — "Write Once, Run Anywhere".

Q6. Correct order of elements in a Java source file: [2019]

- (A) Import, Package, Class
- (B) Class, Import, Package
- (C) Package, Import, Class ✓**
- (D) Any order

Explanation: Always: (1) package (2) import (3) class declaration.

SECTION B — 4-Mark Questions

How is Java strongly associated with the Internet? [2014] (4 Marks)

Ans.

- **Applets** : Small Java programs embedded in web pages via the applet tag.
- **Platform Independence** : Java runs on any OS — Windows, Mac, Linux — critical for web users.
- **Security** : No pointers + bytecode verification prevents web-based attacks.
- **java.net Package** : Provides Socket, URL, URLConnection classes for network programming.
- **Servlets and JSP** : Server-side Java technologies for building dynamic websites.

Conclusion: Java connects to the Internet on both client side (applets) and server side (servlets), making it a complete web language.

What is bytecode? Why is it important? [2017] (4 Marks)

Ans.

- **Definition** : Bytecode is the INTERMEDIATE code produced by the Java compiler (javac).
- **Not Machine Code** : It is NOT native machine code. It is instructions written for the JVM.
- **Storage** : Saved in .class files after compilation.
- **Platform Independence** : The SAME .class file runs on Windows, Mac, Linux — any OS with JVM.
- **Security** : JVM verifies bytecode BEFORE executing it — blocks harmful code.

Program / Syntax:

```
Flow: MyFile.java --> [javac] --> MyFile.class (bytecode) --> [JVM] --> Runs on any OS
```

Conclusion: Bytecode is the reason Java is "Write Once, Run Anywhere".

Explain the role of JVM (Java Virtual Machine). [2024] (4 Marks)

Ans.

- **What it is** : JVM = Java Virtual Machine. The engine that executes Java bytecode.
- **Class Loading** : Loads .class files into memory at runtime.
- **Bytecode Verification** : Checks if bytecode is safe and correct before running.
- **Execution** : Converts bytecode to native machine instructions for the CPU.
- **Memory Management** : Garbage Collector automatically frees unused object memory.
- **Exception Handling** : Manages runtime errors gracefully.

Program / Syntax:

```
JDK = JRE + compiler tools (for DEVELOPERS)
JRE = JVM + libraries (for RUNNING Java)
JVM = the virtual engine (CORE of Java)
```

Conclusion: JVM is the heart of Java. Without JVM, bytecode cannot execute.

How does Java differ from C and C++? [GEN] (4 Marks)

Ans.

- **vs C — Paradigm** : Java is Object-Oriented. C is Procedural.
- **vs C — Pointers** : Java has NO pointers. C uses pointers (risky).
- **vs C — Memory** : Java has automatic Garbage Collection. C needs malloc/free manually.
- **vs C — Platform** : Java is platform-independent. C code needs recompile per OS.
- **vs C++ — Inheritance** : Java: no multiple class inheritance. Uses interfaces instead.
- **vs C++ — Operator** : Java has no operator overloading. C++ has it.
- **vs C++ — GC** : Java: automatic garbage collection. C++: manual memory management.

Conclusion: *Java removed dangerous features of C/C++ (pointers, multiple inheritance) and added safety and portability.*

SECTION C — 12-Mark Questions

Introduction:

Java is a platform-independent, object-oriented language. Every Java program requires JVM to execute.

Simple Java Program:

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
        System.out.println("BCA Sem 3 - AKU Patna");  
    }  
}
```

Explanation of Program Parts:

- **public class HelloWorld** : Class definition. File name MUST match class name.
- **public static void main(String[] args)** : Program entry point. JVM calls this first.
- **public** : Accessible from anywhere.
- **static** : Can be called without creating an object.
- **void** : Does not return any value.
- **String[] args** : Accepts command-line arguments.
- **System.out.println()** : Prints text to screen and moves to next line.

How to Compile and Run:

```
Step 1: javac HelloWorld.java --> creates HelloWorld.class (bytecode)  
Step 2: java HelloWorld --> JVM reads .class and executes it
```

Purpose of JVM:

- **Platform Independence** : Same .class file runs on Windows, Mac, Linux with any JVM.
- **Memory Management** : Garbage Collector frees memory of objects no longer in use.
- **Security** : Bytecode verifier checks code safety before execution.
- **Class Loading** : Loads required classes at runtime as needed.
- **Exception Handling** : Catches and manages unexpected runtime errors.
- **JIT Compilation** : JIT compiler converts frequently-used bytecode to native code for speed.

Conclusion:

JVM is the core of Java's "Write Once, Run Anywhere" philosophy. It transforms bytecode into platform-specific machine code at runtime.

Introduction:

Java is a powerful, general-purpose, object-oriented programming language designed by James Gosling at Sun Microsystems.

Key Features of Java:

- **Simple** : Easy syntax, no pointers, no manual memory management.
- **Object-Oriented** : Supports Encapsulation, Inheritance, Polymorphism, Abstraction.
- **Platform Independent** : "Write Once Run Anywhere" — bytecode runs on any JVM.
- **Secure** : No pointers, bytecode verification, security manager.
- **Robust** : Exception handling, garbage collection, strong type checking.
- **Multithreaded** : Built-in support for concurrent thread execution.
- **Distributed** : java.net for internet and network programming.
- **Portable** : Bytecode is identical on all platforms.
- **High Performance** : JIT compiler speeds up execution at runtime.
- **Dynamic** : Classes loaded at runtime; supports dynamic method dispatch.

Conclusion:

Java's design removes the dangerous parts of C/C++ while adding platform independence, safety, and built-in multithreading support.

UNIT 2 — Constants, Variables & Data Types

Primitive Types · Type Casting · Widening · Narrowing · Scope · Symbolic Constants

SECTION A — MCQ Questions (2 Marks Each)

Q1. Literal can be of which data type? [2018]

- (A) Integer only
- (B) Float only
- (C) Boolean only
- (D) All of the above ✓**

Explanation: Literals can be integers (5), floats (3.14), booleans (true), chars ('A'), strings.

Q2. Which CANNOT be used as a variable name? [2018]

- (A) Identifier
- (B) Keyword ✓**
- (C) Underscore_name
- (D) num1

Explanation: Keywords (int, class, for, etc.) are reserved by Java — cannot be variable names.

Q3. Decrement operator "--" decreases value by? [2018]

- (A) 1 ✓**
- (B) 2
- (C) 3
- (D) 4

Explanation: -- always decreases by exactly 1. Example: int x=5; x--; now x=4.

Q4. Output type of relational operators? [2018]

- (A) Integer
- (B) Boolean ✓**
- (C) Character
- (D) Double

Explanation: All relational operators (>, <, ==, !=, etc.) always return true or false — Boolean.

Q5. Which operator allocates memory for an object? [2018]

- (A) malloc
- (B) alloc
- (C) new ✓**
- (D) create

Explanation: "new" allocates heap memory for an object. No malloc needed in Java.

Q6. Converting double to int is called? [2022]

- (A) Widening
- (B) Narrowing ✓**
- (C) Promotion
- (D) Casting up

Explanation: Narrowing = large to small type. Needs explicit cast. Example: `int x=(int)3.14; x=3.`

SECTION B — 4-Mark Questions

Explain data type conversion and type casting in Java. [2017] (4 Marks)

Ans.

- **Type Conversion** : Changing a value from one data type to another.
- **Widening (Implicit)** : Small to large type. No data loss. Java does it AUTOMATICALLY.
- **Order** : byte short int long float double
- **Example** : `int a = 100; double b = a; // b = 100.0 (automatic)`
- **Narrowing (Explicit)** : Large to small type. Data may be LOST. Must use CAST OPERATOR.
- **Syntax** : `( targetType ) value`
- **Example** : `double pi = 3.14; int x = (int)pi; // x = 3 (loses 0.14)`

Conclusion: Widening = safe, automatic. Narrowing = risky, requires manual cast to avoid compile error.

What is the scope and lifetime of variables in Java? [2018] (4 Marks)

Ans.

- **Scope** : The part of the program where a variable can be accessed and used.
- **Lifetime** : How long a variable stays in memory during program execution.
- **1. Local Variable** : Declared inside a method/block. Scope and lifetime = only that method.
- **2. Instance Variable** : Declared inside class, outside methods. Lifetime = as long as object exists.
- **3. Static Variable** : Declared with "static" keyword. Shared by ALL objects. Lifetime = entire program.

Program / Syntax:

```
class Example {  
    static int classVar = 10; // class scope – shared  
    int instanceVar = 20; // instance scope – per object  
    void method() {  
        int localVar = 30; // local scope – method only  
    }  
}
```

Conclusion: Always declare variables in the smallest scope needed to avoid memory waste.

Define type conversion. Differentiate widening and narrowing. [2022] (4 Marks)

Ans.

- **Type Conversion** : Changing a value from one data type to another compatible type.
- **Widening** : Smaller to larger. NO data loss. Automatic — Java handles it.
- **Narrowing** : Larger to smaller. Data MAY be lost. Manual — needs explicit cast.
- **Rule** : Widening = safe (auto). Narrowing = risky (manual cast required).

Conclusion: Type conversion is fundamental to Java programming. Always think about data loss before narrowing.

What are symbolic constants in Java? How are they defined? **[GEN]** (4 Marks)

Ans.

- **Definition** : Symbolic constants are named values that CANNOT be changed after declaration.
- **Keyword** : Use the "final" keyword to declare a constant.
- **Syntax** : final dataType NAME = value;
- **Example 1** : final double PI = 3.14159; // cannot reassign PI
- **Example 2** : final int MAX = 100;
- **Benefits** : Code is readable (PI vs 3.14159). Change in one place, affects everywhere.

Conclusion: Using constants avoids "magic numbers" in code and makes it easier to maintain.

SECTION C — 12-Mark Questions

Introduction:

Java has 8 primitive data types. They directly store values in memory (not as objects).

Primitive Data Types Table:

Type	Size	Range	Default	Example
byte	1 byte	-128 to 127	0	byte b = 10;
short	2 bytes	-32768 to 32767	0	short s = 1000;
int	4 bytes	-2B to +2B (approx)	0	int i = 500;
long	8 bytes	very large	0L	long l = 123L;
float	4 bytes	7 decimal digits	0.0f	float f = 3.14f;
double	8 bytes	15 decimal digits	0.0	double d = 3.14;
char	2 bytes	Unicode 0 to 65535	\u0000	char c = 'A';
boolean	1 bit	true or false	false	boolean b = true;

Type Casting:

- **Widening (Automatic)** : int x = 100; double d = x; // d = 100.0 (no cast needed)
- **Narrowing (Manual)** : double pi = 3.14; int x = (int)pi; // x = 3 (loses .14!)
- **Cast Syntax** : (targetType) value

Complete Program:

```
public class DataTypeDemo {
    public static void main(String[] args) {
        int age = 21;
        float marks = 85.5f;
        char grade = 'A';
        boolean pass = true;
        double bigMarks = marks; // widening (automatic)
        int approxMarks = (int)marks; // narrowing (manual cast)
        System.out.println("Age: " + age);
        System.out.println("Marks: " + marks);
        System.out.println("Grade: " + grade);
        System.out.println("Pass: " + pass);
        System.out.println("Double: " + bigMarks);
        System.out.println("Int: " + approxMarks);
    }
}
```

Conclusion:

Choose the right data type to save memory. Widening is always safe. Narrowing can lose data
— always use explicit cast.

UNIT 3 — Operators, Branching & Looping

Arithmetic · Relational · Logical · Bitwise · if-else · switch · for · while · do-while

SECTION A — MCQ Questions (2 Marks Each)

Q1. $m = -14$ and $n = -3$. What is $m \% n$? [2014]

- (A) 4
- (B) 2
- (C) -2 ✓**
- (D) None

Explanation: In Java, % takes sign of DIVIDEND. $-14 = (-3)*4 + (-2)$. So $-14 \% -3 = -2$.

Q2. Which operator is used for assignment in Java? [2024]

- (A) ==
- (B) =>
- (C) = ✓**
- (D) :=

Explanation: = is assignment. == is comparison. Java has no => or :=.

Q3. $\text{Math.ceil}(9.7) = ?$ [2014]

- (A) 9
- (B) 9.7
- (C) 10 ✓**
- (D) 11

Explanation: `ceil()` rounds UP to nearest integer. $\text{ceil}(9.7) = 10$. $\text{floor}(9.7) = 9$.

Q4. Which is NOT supported by OOP? [2019]

- (A) Abstraction
- (B) Polymorphism
- (C) Encapsulation
- (D) Global variables ✓**

Explanation: Global variables break encapsulation — a core OOP principle.

Q5. Which loop is guaranteed to run at LEAST ONCE? [GEN]

- (A) for
- (B) while
- (C) do-while ✓**
- (D) All

Explanation: do-while checks condition AFTER the body. So it always runs at least once.

Q6. What does "break" do inside a loop? [2017]

- (A) Pauses the loop
- (B) Restarts the loop
- (C) Exits the loop immediately ✓**
- (D) Skips one iteration

Explanation: break = exits loop completely. continue = skips current iteration only.

SECTION B — 4-Mark Questions

Explain the for loop in Java with an example. [2017] (4 Marks)

Ans.

- **Definition** : A loop used when you know the EXACT number of iterations in advance.
- **Syntax** : for (initialization ; condition ; update) { body }
- **Initialization** : Runs ONCE at the start (e.g., int i = 1)
- **Condition** : Checked BEFORE each cycle. If false, loop stops.
- **Update** : Runs AFTER each cycle (e.g., i++)

Program / Syntax:

```
// Example 1: Print 1 to 5
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}

// Output: 1 2 3 4 5

// Example 2: Sum of 1 to 10
int sum = 0;
for (int i = 1; i <= 10; i++) {
    sum += i;
}

System.out.println("Sum = " + sum); // Sum = 55
```

Conclusion: The for loop is the most commonly used loop when the count is known beforehand.

What is empty statement? Explain its usefulness. [2014] (4 Marks)

Ans.

- **Definition** : An empty statement is just a semicolon (;) with no code inside.
- **What it does** : Does absolutely nothing. It acts as a placeholder.
- **Use 1** : When a loop body is not needed — all work done in the loop header.
- **Use 2** : When an if-else branch needs to be empty.
- **Warning** : Be careful! for(int i=0; i<5; i++) ; — loop body IS the semicolon.

Program / Syntax:

```
for (int i = 0; i < 100; i++) ; // loop runs 100 times, does nothing

if (x > 0) ; // do nothing if positive
else System.out.println("Negative!");
```

Conclusion: Empty statements are rarely used but important to understand to avoid accidental bugs.

Explain break and continue with examples. [2017] (4 Marks)

Ans.

- **break** : Immediately EXITS the loop or switch statement.
- **continue** : Skips the CURRENT iteration only. Loop continues with next.
- **Labeled break** : Used to exit a specific outer loop in nested loops.

Program / Syntax:

```
// break example
for (int i = 1; i <= 10; i++) {
    if (i == 5) break;
    System.out.print(i + " ");
}
// Output: 1 2 3 4

// continue example
for (int i = 1; i <= 10; i++) {
    if (i == 5) continue;
    System.out.print(i + " ");
}
// Output: 1 2 3 4 6 7 8 9 10
```

Conclusion: break = full exit from loop. continue = skip current round, keep looping.

Explain if-else and else-if ladder with examples. [GEN] (4 Marks)

Ans.

- **Simple if** : Executes a block only if condition is TRUE.
- **if-else** : If condition is true — first block. If false — else block.
- **Nested if** : An if inside another if for multi-level decisions.
- **else-if ladder** : Multiple conditions checked one by one from top to bottom.

Program / Syntax:

```
int marks = 75;
if (marks >= 90) System.out.println("A+");
else if (marks >= 80) System.out.println("A");
else if (marks >= 70) System.out.println("B");
else if (marks >= 60) System.out.println("C");
else System.out.println("Fail");
// Output: B (only ONE branch runs)
```

Conclusion: Only ONE block runs in an else-if ladder. Java checks from top to bottom.

SECTION C — 12-Mark Questions

Introduction:

Operators are special symbols that perform operations on values and variables in Java.

Types of Operators:

- **1. Arithmetic** : + - * / % Example: 10+3=13, 10%3=1, 10/3=3 (integer division)
- **2. Relational** : == != > < >= <= Always return Boolean (true/false)
- **Example** : 10 > 3 = true, 10 == 3 = false, 10 != 3 = true
- **3. Logical** : && (AND) || (OR) ! (NOT)
- **&&** : Both must be true. (5>3) && (8>2) = true
- **||** : At least one true. (5>3) || (8<2) = true
- **!** : Reverses boolean. !(5>3) = false
- **4. Assignment** : = += -= *= /= %=
- **Example** : int x=10; x+=5 means x=x+5=15
- **5. Increment/Decrement** : i++ (post: use then add) ++i (pre: add then use)
- **Example** : int x=5; println(x++) prints 5, then x becomes 6
- **6. Bitwise** : & | ^ ~ << >> Work on binary bits of integers
- **Example** : 5(0101) & 3(0011) = 1(0001)
- **7. Ternary** : condition ? value_if_true : value_if_false
- **Example** : int max = (a>b) ? a : b;

Complete Program:

```
public class OperatorDemo {
    public static void main(String[] args) {
        int a = 10, b = 3;
        System.out.println("a + b = " + (a + b)); // 13
        System.out.println("a % b = " + (a % b)); // 1
        System.out.println("a > b : " + (a > b)); // true
        System.out.println("(a>b)&&(b<10): " + ((a>b)&&(b<10))); // true
        System.out.println("a & b = " + (a & b)); // 2
        int max = (a > b) ? a : b;
        System.out.println("Max = " + max); // 10
        a += 5;
        System.out.println("After a+=5: " + a); // 15
    }
}
```

Conclusion:

Operators are the building blocks of every Java program. Mastering all types is essential.

Introduction:

Loops allow code to execute repeatedly. Java provides three types of loop statements.

Loops Overview:

- **1. while Loop** : Checks condition BEFORE each iteration. Runs 0 or more times.
- **Syntax** : while (condition) { body }
- **Use When** : Number of iterations is UNKNOWN in advance.
- **2. do-while Loop** : Checks condition AFTER each iteration. Runs AT LEAST ONCE.
- **Syntax** : do { body } while (condition);
- **Use When** : Must execute at least once (e.g., input validation).
- **3. for Loop** : Best when number of iterations is KNOWN in advance.
- **Syntax** : for (init; condition; update) { body }
- **Use When** : Iterating a fixed number of times.

Comparison Table:

Loop	Check When	Min	Runs	Best Use
while	Before	0	Unknown	iteration count
do-while	After	1	Must run	at least once
for	Before	0	Known	iteration count

Programs:

```
// while: print 1 to 5
int i = 1;
while (i <= 5) { System.out.print(i + " "); i++; }
// Output: 1 2 3 4 5

// do-while: runs once even if condition is false
int n = 10;
do { System.out.println("Runs once: " + n); }
while (n < 5);

// for: sum of 1 to 100
int sum = 0;
for (int j = 1; j <= 100; j++) sum += j;
System.out.println("Sum = " + sum); // Sum = 5050
```

Conclusion:

Choose: for=known count, while=unknown count, do-while=must execute once.

UNIT 4 — Classes, Objects, Methods & Inheritance

Constructor · Method Overloading · Inheritance · Overriding · Abstract · Interface · Access Modifiers

SECTION A — MCQ Questions (2 Marks Each)

Q1. A method that has the SAME NAME as its class is called? [2018]

- (A) Finalize
- (B) Delete
- (C) Class method
- (D) Constructor ✓**

Explanation: Constructor: same name as class, no return type, auto-called when object is created.

Q2. All methods in an abstract class MUST be abstract. [2014]

- (A) True
- (B) False ✓**
- (C) Only in interfaces
- (D) Depends

Explanation: FALSE. Abstract class can have BOTH abstract AND concrete (non-abstract) methods.

Q3. The "new" operator in Java: [2019]

- (A) Returns a pointer
- (B) Creates variable named new
- (C) Obtains memory for a new object ✓**
- (D) Tells available memory

Explanation: "new" allocates heap memory for an object. Example: `Student s = new Student();`

Q4. Superclass of ALL Java classes? [2024]

- (A) Main
- (B) Base
- (C) Object ✓**
- (D) Super

Explanation: `java.lang.Object` is root of all Java classes. Has `toString()`, `equals()`, `hashCode()`.

Q5. Keyword to inherit a class in Java? [2022]

- (A) inherit
- (B) implements
- (C) extends ✓**
- (D) super

Explanation: "extends" for class inheritance. "implements" for interfaces.

Q6. Most restrictive access modifier? [2017]

- (A) public
- (B) protected
- (C) default
- (D) private ✓**

Explanation: private = only same class. public = everywhere. Order: private < default < protected < public.

SECTION B — 4-Mark Questions

★ ★ What is a constructor? What are its special properties? [2014] (4 Marks)

Ans.

- **Definition** : A constructor is a special method used to INITIALIZE an object when it is created.
- **Property 1 — Name** : Same name as the class.
- **Property 2 — No Return Type** : No return type, not even void.
- **Property 3 — Auto-Called** : Called AUTOMATICALLY when object is created with "new".
- **Property 4 — Overloading** : Can be overloaded (multiple constructors with different parameters).
- **Property 5 — Default** : If not defined, Java provides a default no-arg constructor automatically.
- **Type 1 — Default** : Constructor with no parameters.
- **Type 2 — Parameterized** : Constructor with parameters to set initial values.

Program / Syntax:

```
class Student {  
    String name; int roll;  
    Student() { // Default constructor  
        name = "Unknown"; roll = 0;  
    }  
    Student(String n, int r) { // Parameterized constructor  
        name = n; roll = r;  
    }  
}  
  
Student s1 = new Student(); // calls default  
Student s2 = new Student("Zoha", 1); // calls parameterized
```

Conclusion: Constructors ensure objects are always initialized with valid starting values.

Explain constructor overloading with example. [2019] (4 Marks)

Ans.

- **Definition** : Constructor overloading = having multiple constructors in the same class, each with DIFFERENT parameters.
- **How it works** : Java picks the right constructor based on the arguments passed when creating the object.
- **Benefit** : Flexible object creation — different ways to initialize an object.

Program / Syntax:

```
class Rectangle {  
    int length, width;  
    Rectangle() { length = 1; width = 1; } // default  
    Rectangle(int side) { length = side; width = side; } // square  
    Rectangle(int l, int w) { length = l; width = w; } // custom  
    void area() { System.out.println("Area = " + length * width); }  
}  
  
new Rectangle().area(); // Area = 1  
new Rectangle(5).area(); // Area = 25  
new Rectangle(4,6).area(); // Area = 24
```

Conclusion: Constructor overloading makes code flexible without needing separate methods.

★ ★ Differentiate abstract class and interface in Java. [2019, 2024] (4 Marks)

Ans.

- **Abstract Class** : A class declared with "abstract" keyword. Cannot be instantiated directly.
- **Interface** : A contract declared with "interface" keyword. All methods are abstract by default.
- **Methods** : Abstract class: abstract + concrete methods. Interface: all abstract (pre-Java 8).
- **Constructor** : Abstract class: CAN have. Interface: CANNOT have.
- **Inheritance** : Abstract class: extends ONE class. Interface: implements MULTIPLE.
- **Variables** : Abstract class: any type. Interface: only public static final.
- **Use Case** : Abstract class: shared code + related classes. Interface: unrelated classes sharing a behavior.

Conclusion: Rule: If classes are related by nature, use abstract class. If they just share a capability, use interface.

Differentiate extends and implements keywords. [2019] (4 Marks)

Ans.

- **extends** : Used for CLASS inheritance. A subclass extends ONE parent class.
- **implements** : Used when a class implements an INTERFACE. Can implement MULTIPLE interfaces.
- **extends for interface** : One interface can extend another interface using extends.
- **Key Rule** : extends = ONE class only. implements = MULTIPLE interfaces allowed.
- **Relationship** : extends = is-a. implements = can-do.

Program / Syntax:

```
class Dog extends Animal { ... } // Dog IS-AN Animal  
class Duck implements Flyable, Swimmable { ... } // Duck CAN fly AND swim
```

Conclusion: A class can both extend one class AND implement multiple interfaces at the same time.

Explain access modifiers in Java. [2017] (4 Marks)

Ans.

- **Definition** : Access modifiers control WHERE a class member (variable/method) can be accessed.
- **public** : Accessible from ANY class in ANY package.
- **protected** : Accessible in same package AND in subclasses (other packages).
- **default (no keyword)** : Accessible only within the SAME PACKAGE.
- **private** : Accessible ONLY within the same class.

Program / Syntax:

```
// Modifier SameClass SamePackage Subclass OtherPackage  
// public YES YES YES YES  
// protected YES YES YES NO  
// default YES YES NO NO  
// private YES NO NO NO
```

Conclusion: Use private for data (encapsulation). Use public for methods others need to call.

SECTION C — 12-Mark Questions

Introduction:

Inheritance allows a child class to acquire properties and methods of a parent class. It promotes code reuse.

Key Terms:

- **Parent/Superclass/Base Class** : The class being inherited from.
- **Child/Subclass/Derived Class** : The class that inherits. Uses "extends" keyword.
- **super()** : Calls the parent class constructor from child constructor.

Types of Inheritance:

- **1. Single** : One parent, one child. Example: Animal --> Dog
- **2. Multilevel** : Chain of inheritance. Example: Animal --> Dog --> Puppy
- **3. Hierarchical** : One parent, multiple children. Animal --> Dog, Cat, Bird
- **4. Multiple** : Java does NOT support directly. Use interfaces instead.
- **5. Hybrid** : Combination of above types. Achieved via interfaces in Java.

Java Program — Single Inheritance:

```
class Animal {
    String name;

    Animal(String n) { name = n; System.out.println("Animal: " + name); }

    void eat() { System.out.println(name + " is eating"); }

    void breathe() { System.out.println(name + " is breathing"); }

}

class Dog extends Animal {
    String breed;

    Dog(String n, String b) {
        super(n); // call parent constructor
        breed = b;
    }

    void bark() { System.out.println(name + " says: Woof!"); }

    void eat() { System.out.println(name + " eating dog food"); } // override

}

public class InheritDemo {
    public static void main(String[] args) {
        Dog d = new Dog("Tommy", "Labrador");
        d.eat(); // overridden: "Tommy eating dog food"
        d.breathe(); // inherited: "Tommy is breathing"
        d.bark(); // own: "Tommy says: Woof!"
    }
}
```

Output:

```
Animal: Tommy
Tommy eating dog food
Tommy is breathing
Tommy says: Woof!
```

Conclusion:

Inheritance creates "is-a" hierarchies and reduces code duplication. Always call super() first in child constructor.

Introduction:

Object-Oriented Programming (OOP) organizes software around objects. Java supports four core OOP principles.

The Four Pillars:

- **1. Encapsulation** : Bundling data (variables) and methods together inside a class. Hiding data with private.
- **Example** : `class BankAccount { private double balance; public void deposit(double a) { balance += a; } }`
- **Purpose** : Protects data from unauthorized access.
- **2. Inheritance** : Child class acquires properties of parent class. Promotes code reuse.
- **Example** : `class Car extends Vehicle { ... } // Car gets all Vehicle methods`
- **3. Polymorphism** : One name, many forms. Two types: Overloading (compile-time) + Overriding (runtime).
- **Overloading** : Same class, same name, different parameters.
- **Overriding** : Child redefines parent method with same name and parameters.
- **4. Abstraction** : Hiding implementation details, showing only what is needed.
- **Example** : `abstract class Shape { abstract double area(); }`

Conclusion:

Encapsulation=safety, Inheritance=reuse, Polymorphism=flexibility, Abstraction=simplicity.

★ ★ Explain method overloading vs method overriding with examples. [2019, 2020] (12 Marks)

Introduction:

Both involve methods with the same name, but they work in very different ways.

Method Overloading:

- **Definition** : Same class, same method name, DIFFERENT parameters.
- **Resolved at** : COMPILE TIME (static/early binding).
- **Also called** : Compile-time polymorphism.

Method Overriding:

- **Definition** : Parent + Child class, same name, SAME parameters, child redefines behavior.
- **Resolved at** : RUNTIME (dynamic/late binding).
- **Also called** : Runtime polymorphism.

Comparison Table:

Feature	Overloading	Overriding
Class	Same class	Parent + Child
Parameters	DIFFERENT	SAME
Return type	Can differ	Must be same
Resolved	Compile time	Runtime
Type	Static polymorphism	Dynamic polymorphism

Programs:

```
// OVERLOADING example
class Calc {
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }
}

// OVERRIDING example
class Animal { void sound() { System.out.println("Generic"); } }
class Dog extends Animal {
    void sound() { System.out.println("Woof!"); } // overrides
}

Animal a = new Dog();
a.sound(); // Woof! -- runtime decides based on actual object
```

Conclusion:

Overloading = same class, same name, different jobs. Overriding = child redefines parent's behavior at runtime.

UNIT 5 — Arrays, Strings, Multithreading, Exception & Applets

Arrays · Vectors · Interfaces · Thread Lifecycle · Exception · Applet Lifecycle · File I/O

SECTION A — MCQ Questions (2 Marks Each)

Q1. wait() and notify() are defined in? [2014]

- (A) java.lang.String
- (B) java.lang.Runnable
- (C) java.lang.Object ✓**
- (D) java.lang.Thread

Explanation: wait(), notify(), notifyAll() are in java.lang.Object. Every Java object inherits them.

Q2. Runnable interface MUST define which method? [2014]

- (A) start()
- (B) init()
- (C) run() ✓**
- (D) runnable()

Explanation: Runnable has ONE abstract method: run(). Must implement it. Then call thread.start() to launch.

Q3. Which is a valid array declaration in Java? [2014]

- (A) int x();
- (B) float y[]; ✓**
- (C) double () z;
- (D) t int[];

Explanation: float y[]; is valid. Also valid: float[] y; Both are correct syntax.

Q4. Exceptions in Java arise at? [2018]

- (A) Runtime ✓**
- (B) Compile time
- (C) Any time
- (D) None

Explanation: Exceptions are RUNTIME events. They occur while the program is running, not during compilation.

Q5. Thread lifecycle correct order? [2017]

- (A) New->Running->Runnable->Dead
- (B) New->Runnable->Running->Blocked->Dead ✓**
- (C) Running->New->Runnable->Dead
- (D) New->Dead->Running

Explanation: New -> Runnable (start() called) -> Running (CPU assigned) -> Blocked -> Dead (run() ends).

Q6. Which is NOT an Applet lifecycle method? [2020]

- (A) init()
- (B) start()
- (C) paint()
- (D) main() ✓**

Explanation: Applet has: init, start, paint, stop, destroy. NO main() method — that is for standalone apps.

SECTION B — 4-Mark Questions

What is a Vector? How is it different from an array? [2014] (4 Marks)

Ans.

- **Vector Definition** : A Vector is a growable array from the java.util package. Size changes at runtime.
- **Array Definition** : An array has a FIXED size set at creation time. Cannot grow or shrink.
- **Thread Safety** : Vector is synchronized (thread-safe). Array is NOT thread-safe.
- **Data Types** : Array: primitive + objects. Vector: objects only.
- **Methods** : Array: no built-in methods. Vector: addElement, removeElement, size, get, etc.
- **Speed** : Array is faster. Vector is slightly slower due to synchronization overhead.
- **Package** : Array: built-in. Vector: requires import java.util.*;

Program / Syntax:

```
Vector<String> v = new Vector<>();  
v.addElement("Alice"); // add  
v.addElement("Bob");  
System.out.println(v.size()); // 2  
v.removeElement("Alice");  
System.out.println(v.get(0)); // Bob
```

Conclusion: Use Array when size is known and speed matters. Use Vector when size changes dynamically.

How do we tell Java to use a package in a file? [2014] (4 Marks)

Ans.

- **import Statement** : Use the "import" keyword at the TOP of the file to access package classes.
- **Specific import** : import packageName.ClassName; — imports one class only.
- **Wildcard import** : import packageName.*; — imports ALL classes from the package.
- **java.lang** : This package is imported AUTOMATICALLY. No import needed for String, Math, etc.
- **Custom package** : Use "package" keyword at very top to declare your own package.

Program / Syntax:

```
import java.util.Scanner; // import specific class
import java.io.*; // import all from java.io

public class InputDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter name: ");
        String name = sc.nextLine();
        System.out.println("Hello, " + name);
    }
}
```

Conclusion: Always import at top of file, AFTER package statement and BEFORE class declaration.

★ ★ Explain the life cycle of a Thread in Java. [2017, 2018, 2020] (4 Marks)

Ans.

- **Definition** : A thread is the smallest unit of execution. Its lifecycle has 5 states.
- **State 1 — NEW** : Thread object created but start() not yet called.
- **State 2 — RUNNABLE** : start() has been called. Thread is ready, waiting for CPU.
- **State 3 — RUNNING** : CPU is assigned. Thread is executing the run() method.
- **State 4 — BLOCKED/WAITING** : Paused. Waiting for: I/O, sleep(), wait(), or a lock.
- **State 5 — DEAD** : run() method has finished. Thread cannot be restarted.
- **Flow** : NEW --[start()]--> RUNNABLE --[CPU]--> RUNNING --[run() ends]--> DEAD

Program / Syntax:

```
// Key methods:  
start() --> moves thread from New to Runnable  
run() --> contains the thread's actual task code  
sleep(ms) --> pauses thread for given milliseconds  
wait() --> waits until notify() is called  
notify() --> wakes one waiting thread
```

Conclusion: Always call start(), NOT run() directly. start() creates a new thread. run() just runs in current thread.

Define an Applet. How does it differ from an Application? [2017, 2020] (4 Marks)

Ans.

- **Applet** : A small Java program that runs INSIDE a web browser.
- **Application** : A standalone Java program that runs directly in JVM.
- **Entry Point** : Application: main() method. Applet: init() method.
- **Browser** : Application: no browser needed. Applet: requires browser or Applet Viewer.
- **Security** : Application: full access. Applet: restricted (cannot access local files).
- **class extends** : Application: nothing. Applet: extends java.applet.Applet.

Program / Syntax:

```
import java.applet.Applet; import java.awt.Graphics;
// HTML: <applet code="Hello.class" width="300" height="200"></applet>
public class Hello extends Applet {
    public void paint(Graphics g) {
        g.drawString("Hello, BCA!", 50, 50);
    }
}
```

Conclusion: Note: Applets are deprecated in modern browsers. They are still important for AKU exams.

SECTION C — 12-Mark Questions

★ ★ Explain exception handling in Java: try, catch, finally, throw, throws. Write a program. [2014, 2018, 2019, 2020, 2022] (12 Marks)

Introduction:

An exception is an unexpected event during program execution that disrupts normal flow. Java provides structured handling to prevent crashes.

Exception Hierarchy:

- **Throwable** : Root class of all exceptions and errors.
- **Error** : Serious JVM problems. Do not catch. Example: OutOfMemoryError.
- **Exception** : Program-level problems. Should be handled.
- **Checked Exceptions** : Must be handled at compile time. Example: IOException, SQLException.
- **Unchecked/Runtime** : Optional to handle. Example: NullPointerException, ArithmeticException.

Keywords:

- **try** : Block that contains RISKY code that might throw an exception.
- **catch** : Block that HANDLES a specific exception type.
- **finally** : Block that ALWAYS runs, whether exception occurs or not. Used for cleanup.
- **throw** : Manually THROW an exception from inside a method.
- **throws** : Declare in method signature that it MIGHT throw a checked exception.

Complete Program:

```
public class ExceptionDemo {  
    public static void main(String[] args) {  
  
        // Example 1: ArithmeticException  
        try {  
            int a = 10, b = 0;  
            int result = a / b; // throws ArithmeticException!  
            System.out.println(result);  
        } catch (ArithmeticException e) {  
            System.out.println("Error: " + e.getMessage());  
        } finally {  
            System.out.println("finally always runs");  
        }  
  
        // Example 2: Multiple catch blocks  
        try {  
            int[] arr = {1, 2, 3};  
            System.out.println(arr[10]); // index out of bounds!  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Array error: " + e.getMessage());  
        } catch (Exception e) {  
            System.out.println("General error: " + e.getMessage());  
        }  
  
        // Example 3: throw keyword  
        try {  
            int age = -5;  
            if (age < 0)  
                throw new ArithmeticException("Invalid age!");  
        } catch (ArithmeticException e) {  
            System.out.println("Caught: " + e.getMessage());  
        }  
        System.out.println("Program continues normally!");  
    }  
}
```

Output:

```
Error: / by zero  
finally always runs  
Array error: Index 10 out of bounds for length 3  
Caught: Invalid age!  
Program continues normally!
```

Conclusion:

Exception handling keeps programs running gracefully. Always use finally for cleanup. Handle checked exceptions.

★ ★ Explain the Applet Life Cycle with a complete program. [2018, 2019, 2020, 2022, 2024] (12 Marks)

Introduction:

An applet runs inside a web browser. Java automatically calls lifecycle methods at specific moments.

5 Lifecycle Methods:

- **1. init()** : Called ONCE when applet first loads. Use for initialization and setup. (Like constructor)
- **2. start()** : Called after init() AND every time page becomes visible again. Use for starting threads.
- **3. paint(Graphics g)** : Called every time screen needs redrawing. Use for drawing text, shapes, images.
- **4. stop()** : Called when user moves away from the page. Use for pausing animation/threads.
- **5. destroy()** : Called when applet is removed from memory. Use for final cleanup.

Lifecycle Flow:

```
Browser loads page --> init() --> start() --> paint()  
User switches tab --> stop()  
User comes back --> start() --> paint()  
Browser closes --> stop() --> destroy()
```

Complete Program:

```
import java.applet.Applet;
import java.awt.*;

// HTML: <applet code="LifeCycle.class" width="400" height="300">
// <param name="msg" value="AKU BCA Java"></applet>

public class LifeCycle extends Applet {
    String message; int startCount = 0;

    public void init() {
        message = getParameter("msg");
        if (message == null) message = "Hello Applet!";
        setBackground(Color.LIGHT_GRAY);
    }

    public void start() { startCount++; }

    public void paint(Graphics g) {
        g.setColor(Color.BLUE);
        g.setFont(new Font("Arial", Font.BOLD, 20));
        g.drawString(message, 60, 100);
        g.setColor(Color.RED);
        g.drawString("Started: " + startCount, 60, 140);
    }

    public void stop() { System.out.println("stop called"); }
    public void destroy() { System.out.println("destroy called"); }
}
```

Conclusion:

init()=setup once, start()=activate, paint()=draw, stop()=pause, destroy()=cleanup.

★ ★ Explain multithreading in Java. Write programs using Thread class and Runnable interface.

[2014, 2017, 2022] (12 Marks)

Introduction:

Multithreading allows multiple parts of a program to run at the same time. Each part is called a thread.

Why Multithreading?:

- **Better CPU use** : CPU is not idle while waiting for I/O.
- **Non-blocking** : Program does not freeze during slow operations.
- **Parallel tasks** : Download and play music at the same time.

Method 1 — Extend Thread Class:

```
class MyThread extends Thread {  
    String name;  
    MyThread(String n) { name = n; }  
    public void run() { // task code goes here  
        for (int i = 1; i <= 3; i++) {  
            System.out.println(name + ": " + i);  
            try { Thread.sleep(300); } catch (InterruptedException e) {}  
        }  
    }  
}  
  
MyThread t1 = new MyThread("Thread-A");  
MyThread t2 = new MyThread("Thread-B");  
t1.start(); t2.start(); // Both run simultaneously!
```

Method 2 — Implement Runnable (PREFERRED):

```
class MyTask implements Runnable {  
    String taskName;  
    MyTask(String n) { taskName = n; }  
    public void run() { // MUST implement this method  
        for (int i = 1; i <= 3; i++) {  
            System.out.println(taskName + " step " + i);  
            try { Thread.sleep(300); } catch (InterruptedException e) {}  
        }  
    }  
}  
  
Thread t1 = new Thread(new MyTask("Download"));  
Thread t2 = new Thread(new MyTask("Upload"));  
t1.start(); t2.start();
```

Comparison:

- **extends Thread** : Cannot extend another class. Less flexible.
- **implements Runnable** : Can extend another class. More flexible. PREFERRED.

Conclusion:

Use Runnable over extending Thread for better flexibility. Always call start(), not run() directly.

UNIT — MOST REPEATED TOPICS — PYQ ANALYSIS

Topic	Unit	Time s	PYQ Year s	Flag
Exception Handling (try-catch-finally)	Unit 5	5x	★ VVI	2014, 2018, 2019, 2020, 2022
Applet Life Cycle	Unit 5	6x	★ VVI	2017, 2018, 2019, 2020, 2022, 2024
Inheritance (types + Java program)	Unit 4	4x	★ VVI	2017, 2018, 2019, 2020
JVM & Bytecode	Unit 1	3x	★ VVI	2017, 2024, 2025
Multithreading + Thread Life Cycle	Unit 5	3x	★ VVI	2014, 2017, 2022
Constructor & Overloading	Unit 4	4x	★ VVI	2014, 2018, 2019, 2024
OOP Concepts (all 4 pillars)	Unit 4	3x	★ VVI	2018, 2019, 2024
Method Overloading vs Overriding	Unit 4	3x	★ VVI	2017, 2019, 2020
Abstract Class vs Interface	Unit 4	2x	—	2019, 2024
Data Types & Type Casting	Unit 2	3x	—	2017, 2018, 2022
Looping Statements	Unit 3	2x	—	2024, 2025
File I/O (byte vs char streams)	Unit 5	2x	—	2022, 2024
Packages (import, create, use)	Unit 5	2x	—	2014, 2022
Simple Java Program + JVM	Unit 1	2x	—	2024, 2025

UNIT — CHEAT SHEET — LAST MINUTE REVISION

JVM / JRE / JDK

- **JVM** : Java Virtual Machine — runs bytecode, provides platform independence.
- **JRE** : Java Runtime Environment = JVM + standard libraries. Needed to RUN Java.
- **JDK** : Java Development Kit = JRE + compiler tools. Needed to DEVELOP Java.

Applet Lifecycle (A PIE)

- **init()** : Called ONCE on load. Setup here.
- **start()** : Called on each visit. Start threads here.
- **paint()** : Called for drawing. Use Graphics g here.
- **stop()** : Called when page hides. Pause threads.
- **destroy()** : Called on removal. Final cleanup.

Thread Lifecycle

- **NEW** : Thread created. start() not yet called.
- **RUNNABLE** : start() called. Waiting for CPU.
- **RUNNING** : CPU assigned. run() executing.
- **BLOCKED** : Waiting for I/O, sleep, or lock.
- **DEAD** : run() finished. Cannot restart.

OOP Pillars — A PIE

- **Abstraction** : Hide implementation. Show only what is needed.
- **Polymorphism** : One name, many forms (Overloading + Overriding).
- **Inheritance** : Child acquires parent properties. Use extends.
- **Encapsulation** : Bundle data + methods. Use private for data.

Exception Keywords

- **try** : Contains risky code that might throw exception.
- **catch** : Handles a specific exception type.
- **finally** : Always runs — even if exception occurs. For cleanup.
- **throw** : Manually throws an exception inside a method.
- **throws** : Declares that method may throw a checked exception.

Access Modifiers (widest to narrowest)

- **public** : Accessible everywhere — any class, any package.
- **protected** : Same package + subclasses in other packages.

- **default** : Same package only. No keyword needed.
- **private** : Only within the same class. Most restrictive.

Common Mistakes to Avoid

- **Constructor** : Has NO return type — not even void.
- **String compare** : Use `.equals()` not `==` for String comparison.
- **Array index** : Arrays start at index 0 not 1.
- **float literal** : Write `3.14f` not `3.14` for float (`3.14` is double).
- **Multiple inherit** : Java classes cannot extend multiple classes.
- **start vs run** : Always call `start()` to create new thread. NOT `run()`.

Quick Reference

- **Compile** : `javac FileName.java`
- **Run** : `java ClassName`
- **Source extension** : `.java`
- **Bytecode extension** : `.class`
- **Inherit class** : `extends` keyword
- **Use interface** : `implements` keyword
- **Constant** : `final` keyword
- **Shared member** : `static` keyword
- **Object memory** : `new` keyword

All the best for your AKU BCA Sem III Examination!

Study Smart — Focus on VVI Topics — Practice Programs by Hand